

Document Processing Pipeline — Monorepo

View: contracts baked into images

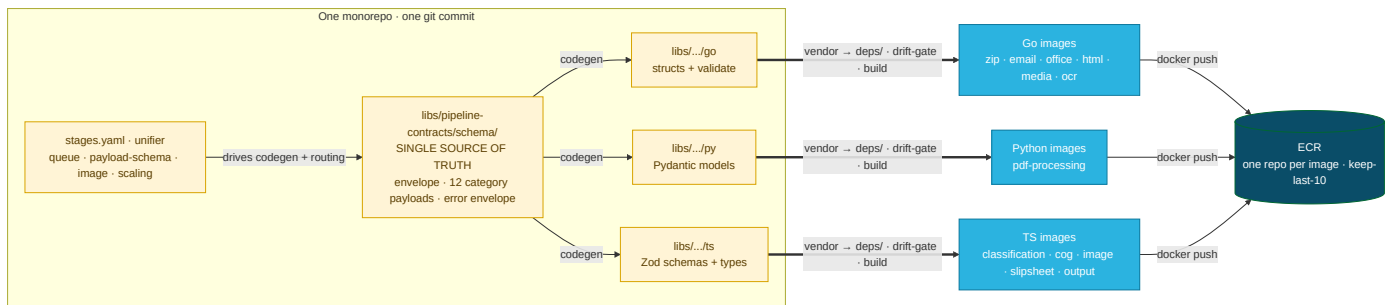
A re-sketch from the **monorepo point of view**. The earlier version drew the contract as a *published, versioned artifact* (@opus2/pipeline-contracts pushed to a private registry) — the **polyrepo / Pact-broker** model. The real project is a **monorepo + microservices** combo, so the contract is *not* a deployable, not published, and has no broker. It is an **internal libs/ package compiled into each service image at build time**; only the **wire envelope** crosses a queue at runtime.

The two things people conflate as "the contract":

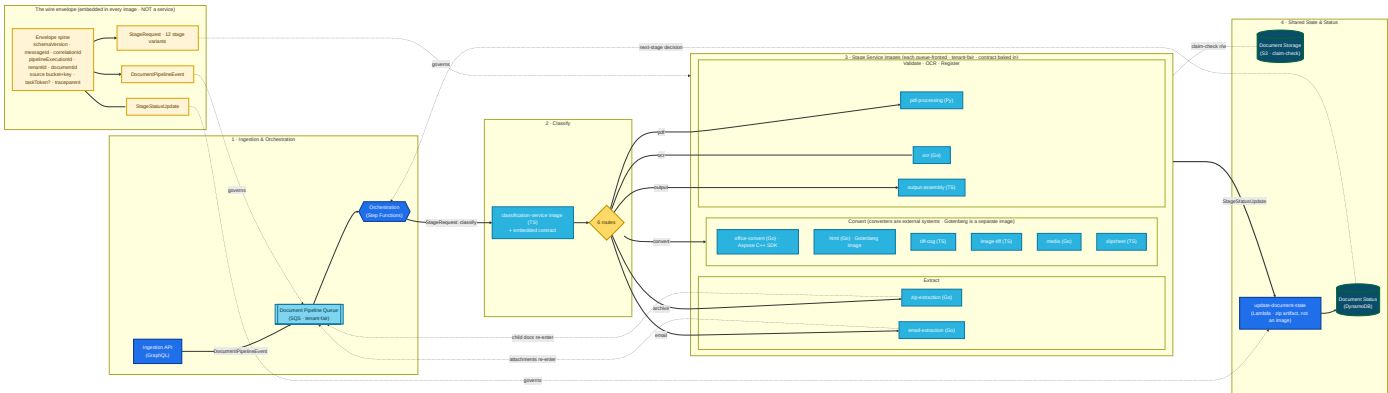
	The contract package	The contract on the wire
What	libs/pipeline-contracts/{go,py,ts} — codegen'd types + validators	The SQS message body: schemaVersion · claim-check · trace context · category payload
When	Build time	Runtime
Where	Source in the repo; baked into every consuming image	Travelling <i>between</i> running Pods, via the queue
Deployable?	No — never an image, ECR repo, or Pod	Not an artifact — it is the <i>agreement</i> the images honor

View 1 · Build time — one repo → one library → many images

The single schema is the source of truth; codegen emits one flavor per language; each flavor is compiled **into** the images of services written in that language; each image is pushed to its own ECR repo. There is no "contract service" — a copy of the contract lives *inside* every consumer.



View 2 · Runtime — images talk; the wire envelope is the only thing that crosses



Same runtime lens as View 2, but tracing a **real document** (not the contract): how the bytes flow through S3 (claim-check) and the stages transform *anything* into a **searchable PDF** returned to the client. Extraction fans children back in; conversion → validation → OCR all converge on Output Assembly.

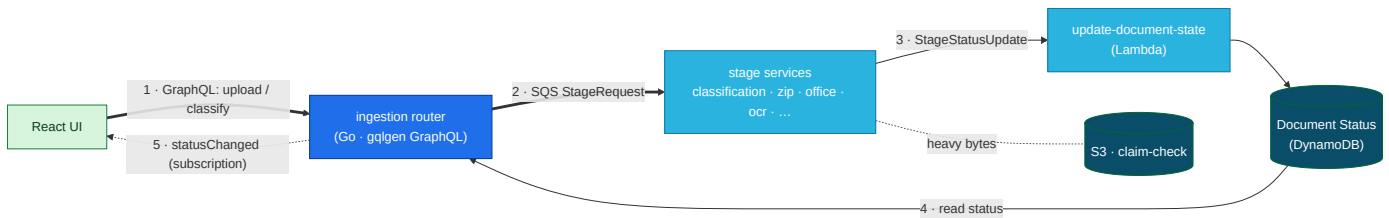


One document's journey through the pipeline. The same **contract sandwich** repeats at every stage: `service-chassis` runs the consume loop, `pipeline-contracts` validates inbound / serializes outbound (with the `schemaVersion` check), `data-access` shapes every DynamoDB / S3-metadata touch, and the **claim-check** keeps heavy bytes in S3. `infra + stages.yaml` shaped the queues, buckets, tables and routing *before* runtime; the three baked libs are enforced *during* it.

View 4 · Contract family + the path into AI-DLC bootstrap

View 5 · Request vs status — the UI only talks to ingestion

A common mix-up: it's **not** "all services → ingestion → UI". The **UI is the entry point** and **ingestion (Go/GraphQL) is the gateway that dispatches out to the stage services**. Services never call ingestion; their status returns via the `update-document-state` Lambda → DynamoDB, which ingestion reads and the UI subscribes to.



Solid ==> (1–2) = request/dispatch, downstream: `UI → ingestion → services`. **--> / -.-> (3–5)** = status/result, upstream via Lambda+DynamoDB back through ingestion to the UI. The UI never talks to a stage service directly.

How to read it

Element	Meaning
View 1 · 2 · 2b · 3 · 4 · 5	Build time (repo → library → vendored into images → ECR); runtime (Pods exchanging the wire envelope); (2b) a real document's journey from upload to final searchable PDF ; execution flow (where each contract is touched); contract family + the path into AI-DLC bootstrap; (5) request vs status — UI ↔ ingestion ↔ services .
Amber <code>WIRE</code> block	The contract is embedded in every image , not a lane you call into. It governs message shape only.
<code>==></code> build edges	<code>codegen → compile into image</code> — the contract is folded into the artifact, not linked at runtime.
Per-image ECR	Each containerized unit → its own image → its own ECR repo, tagged & rolled independently.
<code>- .</code> governs <code>-.-></code>	Ties each message family to the hop it validates: event → queue, request → stages, status → Lambda.
<code>- .</code> <code>claim-check</code> <code>r/w</code> <code>-</code>	Heavy payloads live in S3; only the small envelope (S3 pointer) crosses the wire.

Element	Meaning
<code>- . -></code> dashed	Recursion: zip entries / email attachments re-enter as new documents.
Lambda / Gotenberg notes	Not everything is a built image: Lambdas ship as zip artifacts; Gotenberg is a pulled third-party image.

The point of this version: the contract is reframed as a **build-time internal library baked into ~17 images**, with the **wire envelope** as the only runtime crossing — and `schemaVersion` stays mandatory because a queued message outlives a rolling deploy (old image produced it, new image drains it). Monorepo ⇒ plain import, no broker; the "rebuild all consumers" path-filter rule keeps every embedded copy consistent. This is the **monorepo + microservices** picture: one repo, many images, one contract embedded everywhere.